

Module 11 — Question Pool

OMSCS 6250 Computer Networks

Lesson 11: Applications — Video & Multimedia

Multimedia Applications: A Perspective From the Network

Q1. [MCQ] Multimedia vs. bulk file transfer

From the network's perspective, what makes MULTIMEDIA applications fundamentally different from bulk file transfer?

- **A.** They have strict timing constraints (delay, jitter) and can often tolerate some loss — the opposite trade-off from file transfer, which tolerates delay but requires perfect reliability.
- **B.** They use no transport-layer protocol; they speak directly to IP, so the network must reserve a dedicated per-flow circuit and cannot statistically multiplex their packets with ordinary datagram traffic.
- **C.** They are stateless at the network layer, while file transfer is stateful.
- **D.** They require BGP communities to identify their priority class along the path.

Correct answer: **A**

Why: Multimedia: tight delay + loss-tolerant (opposite of file transfer). Voice/video need every byte to arrive ON TIME more than to arrive at all; file transfer can wait but cannot drop bytes.

Types of Multimedia Applications and Characteristics

Q2. [TF] Live streaming vs. conversational VoIP delay-sensitivity

Streaming live audio/video is generally LESS delay-sensitive than conversational VoIP, because some buffering is tolerated by live viewers but not by parties in an interactive call.

- True
- False

Correct answer: **True**

Why: Live streaming > conversational VoIP in delay tolerance. Live viewers accept a few-second lag; interactive call participants don't tolerate even 400 ms without obvious disruption.

How Does VoIP Work?

Q3. [MCQ] Why VoIP uses UDP instead of TCP

Why is the UDP protocol, rather than TCP, popular for VOIP applications?

- **A.** TCP cannot traverse NAT, but UDP can without any helper protocol, because UDP writes its public address into the checksum field so NAT rewrites it automatically, while TCP's three-way handshake is silently dropped at the NAT box.
- **B.** UDP guarantees in-order, lossless delivery — exactly what voice requires.
- **C.** UDP has built-in encryption that VoIP requires for E911 compliance.
- **D.** TCP retransmissions arrive too late to be useful for real-time audio, and TCP's congestion-control rate cuts on loss can drop the sender below the receiver's playout rate — both kill the call experience.

Correct answer: **D**

Why: Retransmits too late + cwnd cuts on loss = wrong for voice. TCP would deliver yesterday's audio sample after RTT delay, and its loss-driven rate cuts can starve the codec; UDP keeps the stream flowing.

QoS for VoIP: Metrics

Q4. [MCQ] The three VoIP QoS metrics

Which set of metrics is most directly relevant to VoIP quality of service?

- **A.** End-to-end delay, delay jitter, and packet loss — all three together affect quality of service.
- **B.** Only end-to-end delay matters; jitter and loss are not perceptible.
- **C.** Throughput (Mbps) and packet count per second; nothing else matters for human voice.
- **D.** BGP path length and TCP congestion-window size.

Correct answer: **A**

Why: VoIP metrics = delay + jitter + loss (all three matter). Each impairs intelligibility differently: long delay kills interactivity, jitter forces playout buffer, loss creates gaps unless concealed.

QoS for VoIP: Delay Jitter

Q5. [MCQ] Purpose of the jitter (play-out) buffer

What is the main purpose of a JITTER (play-out) BUFFER at the VoIP receiver?

- **A.** To perform forward error correction across packets, rebuilding lost samples from parity bytes in each packet header before decoding.
- **B.** To compress audio before decoding it for the user.
- **C.** To smooth out variation in per-packet arrival delay so the application can play out audio at a steady rate without gaps.
- **D.** To negotiate the codec with the sender.

Correct answer: **C**

Why: Jitter buffer smooths variable arrival => steady playout. Packets arrive with variable network jitter; the buffer absorbs the variation so the playout clock can advance at a constant rate without underruns.

Q6. [MCQ] The jitter-buffer length trade-off

What is the core TRADE-OFF in choosing the LENGTH of a jitter buffer?

- **A.** Buffer length has no impact on either metric in modern codecs.
- **B.** Buffer length affects loss but not delay.
- **C.** A longer buffer reduces BOTH delay and loss simultaneously, therefore a longer buffer is more optimal.
- **D.** A longer buffer reduces dropped/late packets but adds end-to-end delay; a shorter buffer cuts delay but increases late/dropped packets.

Correct answer: D

Why: Longer buffer = fewer late/dropped, more delay; shorter = opposite. The fundamental trade-off in jitter buffer sizing — operators tune it to balance perceived lag against gap frequency.

QoS for VoIP: Packet Loss

Q7. [MCQ] What counts as a lost packet in VoIP

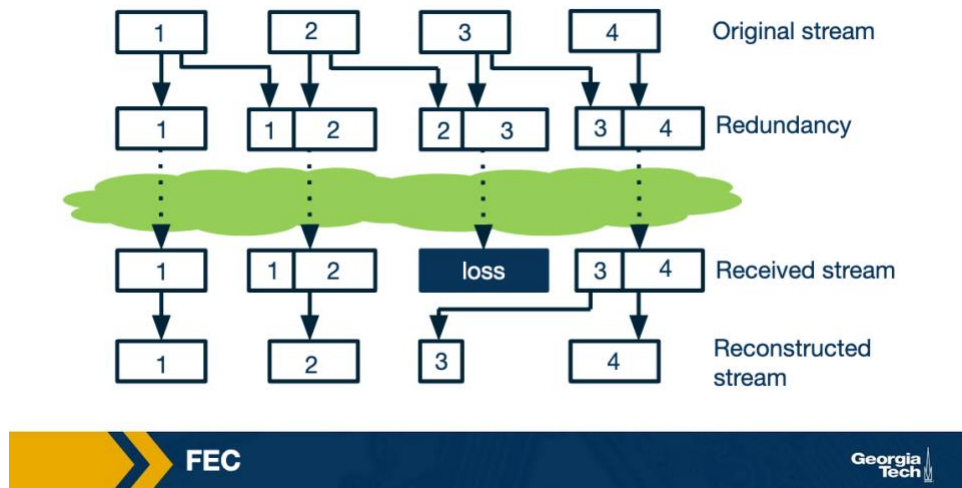
In VoIP, a packet is considered LOST under which condition(s)?

- **A.** The packet never arrives OR the packet arrives after its scheduled playout time — both count as lost.
- **B.** Only if the packet fails to arrive; late packets are still useful.
- **C.** Only if the packet's payload checksum fails at the receiver.
- **D.** Only if more than three consecutive packets are missing in a row.

Correct answer: A

Why: Lost = never arrived OR arrived too late for playout. Beyond the playout deadline, a packet is useless — it's effectively lost even if technically received.

Q8. [MCQ] FEC vs. retransmission for VoIP loss



Reference Figure from Canvas Module: VoIP — forward error correction on lossy paths (Module 11)

Consider the following scenario: a VoIP path experiences occasional bursts of loss. Why is Forward Error Correction (FEC) often preferred over retransmission for these losses?

- **A.** FEC requires no extra bandwidth and is free at the encoder side, because the repair data is folded into unused padding bits of the existing packets rather than sent as additional traffic.
- **B.** FEC reconstructs missing audio from redundant information sent alongside the original packets — recovery is immediate at the receiver, with no extra RTT for retransmission.
- **C.** FEC eliminates the need for a jitter buffer.
- **D.** Retransmission is generally cheaper for the codec than FEC.

Correct answer: **B**

Why: FEC = redundancy sent ALONGSIDE => recover at receiver, no extra RTT. Encoder pre-computes recovery info; receiver reconstructs missing audio from the redundancy without ever asking for retransmission.

Q9. [MCQ] Interleaving for packet-loss mitigation

How does INTERLEAVING mitigate audio packet loss, and what is its main trade-off?

- **A.** It sends a redundant low-quality copy of each chunk alongside the original, so a lost chunk is reconstructed immediately; the trade-off is the extra bandwidth the redundancy consumes.
- **B.** It reorders audio chunks before sending so a single lost packet is spread into several no-consecutive small gaps instead of one large gap, adding no redundant data; the trade-off is added latency, which limits its usefulness for real-time VoIP.
- **C.** It requests retransmission of only the lost chunks so the stream is rebuilt perfectly; the trade-off is one extra round-trip of delay per loss.
- **D.** It raises the encoding bitrate whenever loss is detected so each packet carries more samples; the trade-off is higher CPU load at the encoder.

Correct answer: **B**

Why: Interleaving adds NO redundancy (unlike FEC) and does NOT retransmit. It reorders chunks so that a burst loss becomes several small, non-consecutive gaps -- many small gaps are less disruptive than one large one (the ear tolerates gaps under ~30ms). Cost: the receiver must wait for interleaved chunks to be reassembled, which adds latency -- so it suits streaming stored audio better than interactive VoIP.

Q10. [TF] Real-time audio recovery: FEC or concealment

In real-time audio, retransmissions arrive too late to be useful — recovery must come from redundancy already in the stream (FEC), or from concealing the loss at the decoder.

- True
- False

*Correct answer: **True***

Why: Real-time audio cannot use retransmits => FEC or concealment. Retransmission RTTs blow the playout deadline; the only viable recovery is data the receiver already has (FEC) or hiding loss in the decoder (PLC).

Live/On Demand Streaming Introduction

Q11. [MCQ] Live vs. on-demand streaming

What is the main difference between LIVE and ON-DEMAND streaming, from a buffer and transport perspective?

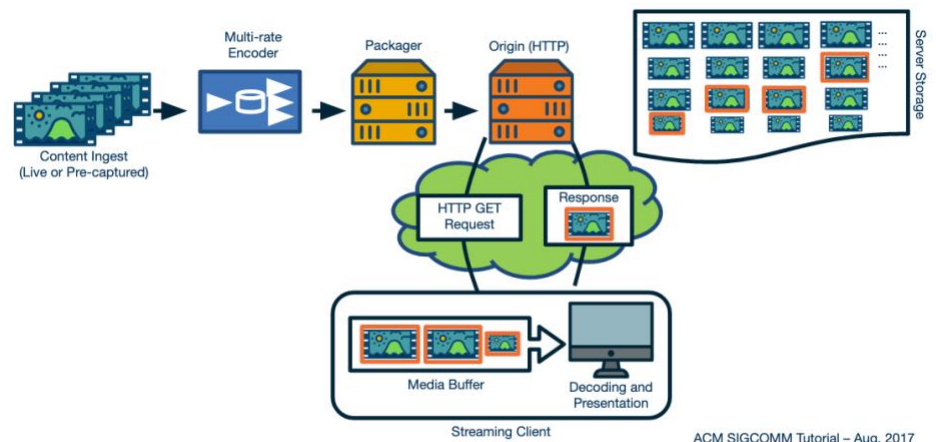
- **A.** On-demand requires DRM and live does not.
- **B.** Live uses UDP and on-demand uses TCP, and that is the only difference; the receiver keeps a fixed one-second buffer in both cases, so transport choice alone fixes all timing.
- **C.** On-demand can pre-buffer many seconds and tolerate large startup delays; live has a much tighter end-to-live budget and can pre-buffer only a few seconds.
- **D.** Live generally uses higher bitrates than on-demand.

*Correct answer: **C***

Why: On-demand: large pre-buffer; live: tight end-to-live budget. On-demand can buffer many seconds with no UX cost; live has a small end-to-live window because the content is being produced as it's consumed.

Video Streaming Bigger Picture

Q12. [MCQ] The end-to-end video streaming pipeline



Adaptive Streaming over HTTP



Reference Figure from Canvas Module: End-to-end video pipeline — capture, encode, host, deliver, decode, render (Module 11)

From the bigger-picture figure, which sequence orders the video streaming pipeline correctly from creation to playback?

- A. DRM → Encode → Capture → Render → Decode.
- B. Encode then Capture then DRM then CDN host then Client then Decode then Render, compressing before capture.
- C. Capture → Render → Encode → DRM → Decode.
- D. Capture (raw) → Encode/compress → DRM/host on CDN → Client downloads over Internet → Decode → Render.

Correct answer: **D**

Why: Capture -> Encode -> DRM/CDN host -> Client download -> Decode -> Render. The full pipeline from camera to viewer's screen — each stage is its own service in a modern streaming system.

UDP vs TCP

Q13. [MCQ] Why not UDP for stored video (I-frame dependency)

Consider the advantages and disadvantages of using UDP for stored video streaming. What is the main argument AGAINST using UDP for stored video streaming?

- A. UDP headers are larger than TCP headers.
- B. UDP is blocked by all CDNs and cannot reach end users.
- C. UDP cannot carry compressed video, because its 8-byte header cannot mark I-frame boundaries, so a compressed stream is rarely resynchronized at the decoder.
- D. Losing part of an I-frame can prevent decoding of the frames that depend on it — reliable delivery (TCP) avoids that catastrophic outcome on stored content.

Correct answer: **D**

Why: UDP loss in an I-frame => dependent frames undecodable; TCP avoids this on stored video. Bulk pre-recorded content can wait for retransmission; the catastrophic cost of losing reference frames outweighs the latency cost of TCP.

Why Do We Use HTTP?

Q14. [MCQ] Why providers use HTTP for video delivery

Why do content providers prefer using HTTP for video delivery over designing a custom streaming protocol?

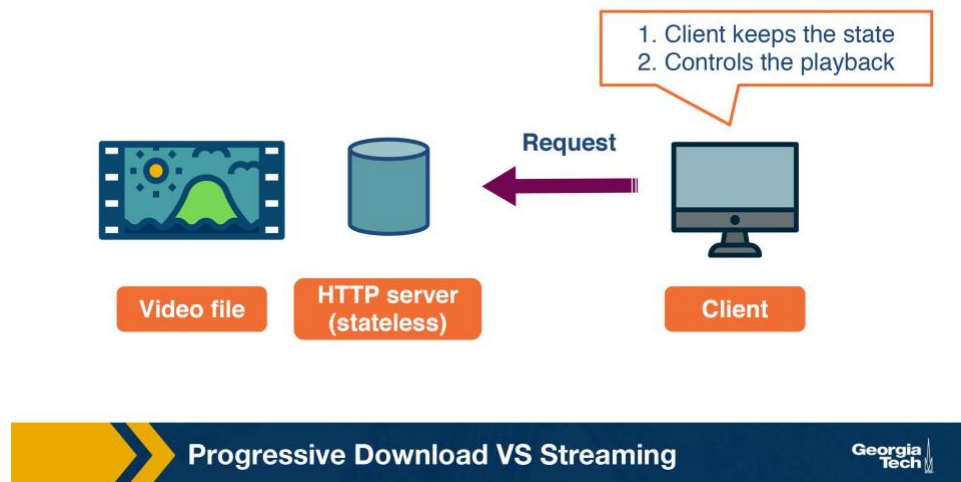
- **A.** HTTP streaming reuses existing CDN infrastructure and passes easily through middleboxes/firewalls; that operational fit outweighed efficiency gains a custom protocol might offer.
- **B.** HTTP guarantees lower end-to-end latency than any custom protocol can achieve.
- **C.** HTTP is the only protocol that can run over TCP across the Internet.
- **D.** HTTP servers keep per-client state, simplifying adaptation by maintaining a server-side model of each viewer's buffer occupancy and choosing the next segment's bitrate on the client's behalf.

Correct answer: **A**

Why: HTTP streaming = reuses CDN + middlebox-friendly => operational win. Custom protocols would have required new CDN deployments and firewall traversal; HTTP just worked on existing infrastructure.

Progressive Download vs Streaming

Q15. [MCQ] Progressive download vs. paced streaming



Reference Figure from Canvas Module: Progressive download vs paced streaming (Module 11)

Two ways to fetch a stored video over HTTP are (a) progressive download -- a single HTTP GET in which the server pushes the file as fast as TCP allows -- and (b) paced (chunked) streaming, in which the client keeps only a few seconds of video in a playout buffer and requests more content as it is consumed. After

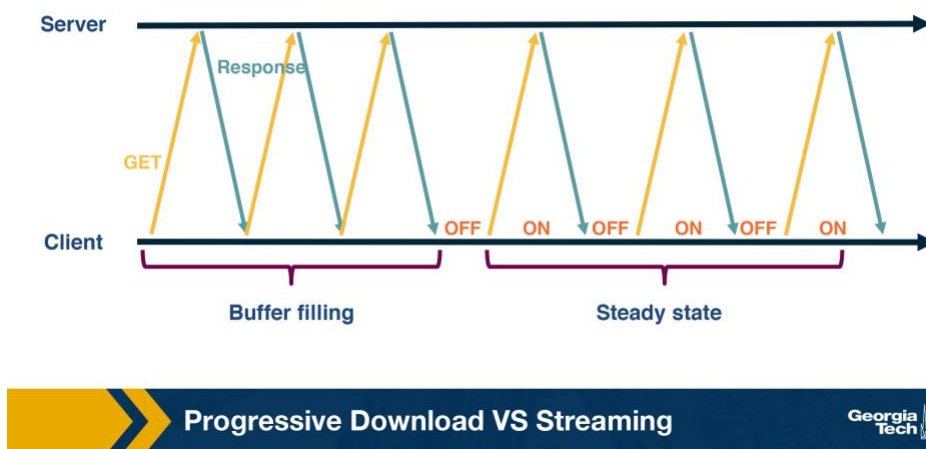
startup, once the buffer has filled, what distinguishes the STEADY STATE of paced (chunked) streaming from progressive download?

- **A.** Progressive download uses UDP while paced streaming uses TCP; because UDP has no flow control, the progressive client cannot pace its requests and tends to drain the server at line rate regardless of buffer.
- **B.** Paced streaming follows an ON-off pattern — the client waits for the buffer to deplete below a threshold, then requests more content; progressive download just fills as fast as TCP can.
- **C.** Paced streaming has no buffer; progressive download has a buffer.
- **D.** Both are identical at steady state; only startup differs.

Correct answer: **B**

Why: Paced streaming = ON-OFF (wait for buffer threshold); progressive = fill as fast as TCP allows. Paced lets the client request only when needed; progressive downloads everything upfront, even content the user may never watch.

Q16. [MCQ] TCP consequence of the OFF period



Reference Figure from Canvas Module: ON-OFF pattern in paced streaming (Module 11)

In the steady state of paced (chunked) streaming, the client fills its playout buffer and then STOPS requesting data until the buffer drains below a threshold -- the idle gap between chunk downloads is what produces the ON-OFF pattern shown in the reference figure. Consider one such cycle: the buffer fills, the client pauses requests during the OFF period, then resumes and downloads the next chunk over the SAME TCP connection. What is a CONSEQUENCE of that idle OFF period for TCP, and why does it matter for a throughput-based bitrate estimate?

- **A.** TCP automatically doubles its rate during OFF periods to compensate.
- **B.** TCP keeps its prior congestion window forever, so throughput is unaffected, because the kernel freezes cwnd while the socket is idle and restores it for the next chunk.
- **C.** TCP's congestion window may reset during the idle off period, so the next chunk's measured throughput is biased low relative to true link capacity.
- **D.** UDP takes over and the throughput becomes constant during OFF periods.

Correct answer: **C**

Why: OFF period => TCP cwnd may RESET => measured throughput biased LOW. Many TCP implementations slow-start after an idle period; the next ON measurement starts with a small cwnd and may finish before reaching the real link capacity.

How to Handle Network and User Device Diversity?

Q17. [MCQ] How DASH handles device and network diversity

DASH (Dynamic Adaptive Streaming over HTTP) addresses device and network diversity in what way?

- **A.** DASH delivers the same single bitrate to every user and lets the codec scale the visual quality.
- **B.** DASH transcodes the video on the fly for each individual user.
- **C.** DASH uses a different TCP variant for every device class.
- **D.** The server stores the video as segments, each encoded at multiple bitrates; the client picks a bitrate per segment based on current conditions.

Correct answer: **D**

Why: DASH = segments x bitrates; client picks per segment. The video is pre-encoded at multiple bitrates and chopped into segments; the client switches bitrate granularly based on current network/buffer conditions.

Q18. [TF] The DASH manifest file

In DASH, the client downloads a MANIFEST at the start of the session that lists the available bitrates and the URLs for each segment.

- True
- False

Correct answer: **True**

Why: Manifest lists bitrates + URLs upfront. The MPD (Media Presentation Description) is fetched first; it tells the player what bitrates exist and where to fetch each segment from.

Bitrate Adaptation in DASH

Q19. [MCQ] What bitrate adaptation does

At a conceptual level, what does BITRATE ADAPTATION in DASH do during playback?

- **A.** The origin server re-encodes the video to a new bitrate for each viewer in real time and pushes it at a fixed rate.
- **B.** The bitrate is chosen once when the session starts and then held constant for the whole video, no matter how the network changes.

- **C.** The client keeps re-selecting which quality (bitrate) version of the upcoming segments to request, so the delivered video quality tracks changing network conditions instead of staying fixed.
- **D.** The client always requests the single highest bitrate and relies on TCP retransmissions to make up any shortfall in bandwidth.

Correct answer: **C**

Why: Bitrate adaptation = the client keeps re-picking segment quality over time to match current conditions. Quality is dynamic and client-driven -- the goal is to keep quality as high as the network allows while avoiding stalls, not to lock one bitrate for the whole session.

What are the Goals of Bitrate Adaptation?

Q20. [MCQ] The three goals of bitrate adaptation

DASH bitrate adaptation is primarily driven by THREE possibly-conflicting goals. Which list is correct?

- **A.** Maximize quality, minimize stalls, and maximize the number of bitrate switches per second, treating frequent switching as a benefit that keeps the encoder calibrated.
- **B.** Maximize quality (highest sustainable bitrate), minimize stalls (don't deplete the buffer), and minimize quality switches (smooth user experience).
- **C.** Maximize quality, maximize energy use, and minimize delay.
- **D.** Minimize quality, minimize stalls, and minimize delay.

Correct answer: **B**

Why: Maximize quality + minimize stalls + minimize switches. Three conflicting goals: higher bitrate = better quality but risks stalls; switching often = responsive but jarring; smoothing = comfortable but slow to react.

Bitrate Adaptation Algorithms

Q21. [MCQ] Throughput-based vs. buffer-based adaptation

Bitrate adaptation algorithms split broadly into THROUGHPUT-based and BUFFER-based families. What is the core idea of each?

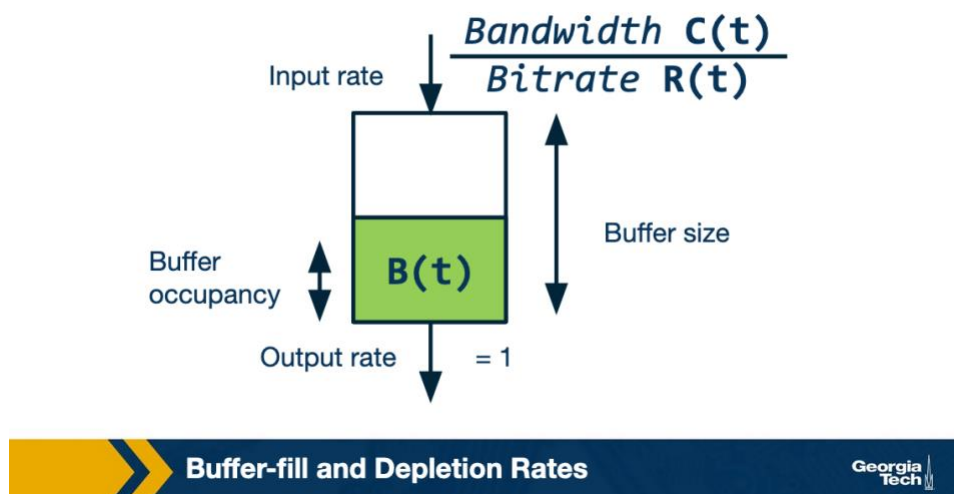
- **A.** Throughput-based and buffer-based are identical; the names are vendor labels.
- **B.** Throughput-based: count packets and pick a bitrate proportional to packet rate. Buffer-based: ignore everything and just pick the lowest bitrate.
- **C.** Throughput-based: estimate available bandwidth and pick the highest bitrate $\leq$ that estimate. Buffer-based: pick bitrate as a function of current buffer occupancy, ignoring throughput estimates.
- **D.** Both algorithms use only the receive window (rwnd) of TCP as input.

Correct answer: **C**

Why: Throughput-based: estimate BW, pick highest bitrate $\leq$ estimate. Buffer-based: bitrate is $f(\text{buffer occupancy})$, ignoring throughput. Two families with different blind spots and trade-offs.

Throughput-Based Adaptation and its Limitations

Q22. [MCQ] Why throughput-based adaptation stalls after a bandwidth drop



Reference Figure from Canvas Module: Throughput-based adaptation under sudden bandwidth drop (Module 11)

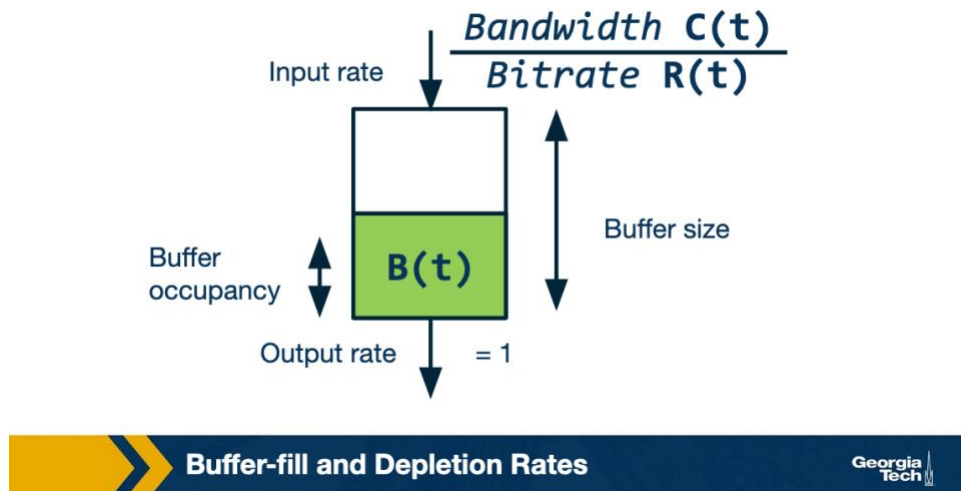
Recall that a throughput-based player chooses the bitrate of the NEXT chunk from the throughput it measured on RECENT, already-downloaded chunks, and that its playout buffer fills as chunks arrive but drains at real-time playback speed. Consider such a player just after the available bandwidth SUDDENLY drops. Conceptually, why can throughput-based adaptation cause the video to STALL right after that drop, even when a lower bitrate that would fit the new bandwidth was available?

- **A.** It stops requesting any new chunks whenever bandwidth changes, so playback halts by design until the user restarts the stream.
- **B.** It sizes its next request from recently measured (pre-drop) throughput, so it can request a chunk far too large for the now-reduced bandwidth; that chunk then arrives more slowly than the video plays, and the buffer can drain to empty before it finishes.
- **C.** A bandwidth drop forces the player onto UDP, which has no congestion control, so packets are lost and the picture freezes.
- **D.** Lower bandwidth raises the video's encoded bitrate, so every chunk becomes larger and playback can never catch up.

Correct answer: **B**

Why: Throughput-based adaptation is backward-looking: it picks the next bitrate from past throughput. A sudden drop isn't noticed until an over-sized chunk is already in flight -- if that chunk can't arrive before the buffer empties, the player stalls.

Q23. [MCQ] Buffer behavior during a slow chunk download



Reference Figure from Canvas Module: Buffer depletion during the over-estimated chunk download (Module 11)

While a throughput-based player is waiting on a chunk that is downloading more slowly than the video plays, what happens to the playout buffer, and why?

- **A.** It drains -- buffered video is played out (and removed) faster than the slow chunk can refill it, so the buffer shrinks toward empty and the player risks re-buffering.
- **B.** It stays constant, because the player pauses the playout clock while any download is in flight, so no buffered video is consumed.
- **C.** It grows, because having a download in progress always adds more to the buffer than playout removes.
- **D.** It is unaffected, because the buffer only changes when the user manually pauses or seeks.

Correct answer: **A**

Why: Playout keeps consuming buffered video at real-time speed; if the incoming chunk arrives slower than that, refill can't keep up and the buffer heads toward empty -- the classic re-buffering setup.

Q24. [TF] Buffer drain rate is independent of bitrate

In the throughput-based adaptation model, the playout buffer drains at the rate the video is consumed -- one second of buffered video removed for each second of real-time playback -- regardless of the encoding bitrate.

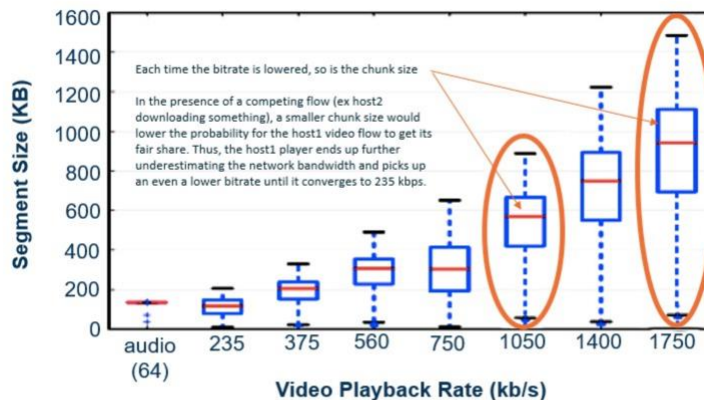
- True
- False

Correct answer: **True**

Why: The drain rate is set by playback, not by bandwidth or bitrate. A higher-bitrate stream has larger chunks but is still played out at real-time speed, so the buffer empties at the pace of playback, independent of the chosen bitrate.

Rate-based Adaptation Mechanisms

Q25. [MCQ] Self-reinforcing bandwidth under-estimation



Problem of Underestimation

Georgia Tech

Reference Figure from Canvas Module: Bandwidth UNDER-estimation with small chunks (Module 11)

A DASH player streaming over a shared link competes with a long-lived TCP flow (say, a bulk file download). In principle TCP should split the link roughly fairly, so the player could sustain a bitrate near its fair share -- yet in practice it drifts DOWN to a much lower bitrate than that share, sometimes all the way to the minimum. Two facts matter: the player requests one chunk at a time with idle gaps between requests (the ON-OFF pattern), and lower bitrates produce physically smaller chunks. Conceptually, why does this make the player under-estimate its fair-share bandwidth AND keep reinforcing that under-estimate downward?

- **A.** TCP is specified to give long-lived bulk transfers priority over video, so the standard deliberately starves DASH flows to a fraction of their fair rate.
- **B.** A competing UDP flow crowds the DASH client out and forces it to the lowest bitrate.
- **C.** The manifest removes the higher bitrates from the client's choices as soon as any competing flow appears.
- **D.** Low-bitrate segments are small, so each download finishes before TCP ramps up to its fair share; the client measures an artificially low throughput, then requests an even lower bitrate, whose still-smaller chunk confirms the low estimate -- a self-reinforcing downward loop.

Correct answer: **D**

Why: Small chunks end before TCP reaches its fair-share rate, so the throughput sample is biased low; the client then asks for an even lower bitrate, whose chunks are smaller still -- the estimate spirals downward instead of converging to fair share.

Q26. [MCQ] Why ON-phase throughput is biased low

In the steady state, a DASH client alternates ON periods (actively downloading the next chunk) with OFF periods (idle, because the playout buffer is full and it is waiting for the buffer to drain). It estimates the available bandwidth from the throughput it measures DURING an ON period. But after an idle OFF period,

TCP has been sitting idle and its congestion window has decayed back toward its initial small value. Why does this make the ON-period throughput measurement **BIASED LOW** relative to the link's actual capacity?

- **A.** UDP traffic from other applications saturates the link only during OFF periods, and that cross traffic lingers into the next ON phase, so the first samples read a congested link.
- **B.** TCP becomes faster after every OFF period, so on-period throughput overshoots.
- **C.** TCP may reset cwnd during the off period, so the next ON starts with a small cwnd — and may end before cwnd has ramped back up to the real link capacity.
- **D.** The manifest forbids the player from using high bitrates during ON periods.

Correct answer: **C**

Why: TCP cwnd may RESET during OFF => next ON starts with small cwnd => measurement biased low. The very mechanism that makes paced streaming polite to the network breaks throughput estimation.